Mega-Trading: Generative Order-Flow Modeling from Public Market Data

Mega-Trading Project

Submitted take-home project

Abstract

Mega-Trading is a take-home prototype for modeling market-event sequences with the same basic idea used by language models: predict the next token from recent context. The project uses public Binance historical trade data, converts each trade event into a discrete event token, trains a compact decoder-only Transformer inspired by Llama-style language models, and evaluates the model on a chronological holdout set. The design is motivated by recent work on financial-market foundation models such as TradeFM, but this implementation is intentionally smaller, reproducible, and based only on public data [2, 3, 6, 4].

1 System Overview

The goal is to make the project easy to inspect. Public trade data is converted into a sequence of simple event tokens. The model learns to predict the next event token, and each generated token can be decoded back into market-event features such as side, relative price depth, size bucket, and elapsed time. This keeps the prototype close to the "next token prediction" pattern used in LLMs while staying interpretable for market data [5, 6].

2 Take-Home Scope

The interview prompt was intentionally open-ended: build a non-trivial system, choose the problem, decide how far to take it, define success criteria, and make the work reviewable through version control. I used that prompt to build an end-to-end system for generative market-event modeling rather than a single notebook or isolated model experiment.

- **Problem choice**: model public market events as a token sequence, inspired by the way LLMs learn next-token prediction.
- **Data system**: download public Binance trade data, normalize it into event fields, fit a tokenizer, and create chronological train, validation, and backtest splits.
- **Model and training system**: implement a decoder-only Transformer, train it with mixed precision, track metrics with MLflow, and optimize throughput with preloaded NumPy streams, `torch.compile`, and a Triton attention path.
- **Evaluation**: report held-out loss, perplexity, top-k accuracy, simple baselines, and generated vs. real event statistics.
- **System delivery**: package the checkpoint context, tokenizer metadata, reports, diagrams, and a Hugging Face Static Space page so the work can be reviewed end to end.
- **Version control**: keep configs, source modules, tests, and generated showcase artifacts separated so each part of the system is reviewable as a focused diff.

3 Motivation and Fit

I chose this project because it connects the company's market-data focus with my own background as a research engineer building end-to-end foundation-model systems. Deeter Analytics describes its mission as turning complex market data into actionable insight with clarity, precision, foresight, and transparent methodology [1]. A generative market-event modeling system is a natural take-home problem under that context: it starts from raw market data, builds a model-visible representation, trains a sequence model, evaluates outputs against held-out data, and packages the evidence for review.

My current work is on Amazon Nova foundation models, where the research engineering role is to partner with researchers and help move model ideas from experiments into production systems. Mega-Trading reflects that same operating style at take-home scale: define a research-shaped problem, build the data and training pipeline around it, optimize the system enough to run real experiments, and present success criteria and limitations honestly.

4 Data and Event Representation

The data source is Binance's public historical trade archive, which can be downloaded without private exchange credentials [2]. Each raw trade is mapped into a compact event representation:

- `action`: simplified event type used by the model.
- `side`: buyer- or seller-initiated trade direction.
- `relative_price_bps`: log order price relative to estimated midprice.
- `price_depth_bps`: absolute relative price depth.
- `size`: causally normalized relative size.
- `interarrival_seconds`: elapsed time since the prior event for the ticker.

The submitted dataset artifacts contain 62,012,828 token windows across 32 USDT symbols and 1,096 symbol-month partitions from 2023-05 to 2026-04. The chronological split contains 54,695,345 train windows, 1,116,216 validation windows, and 6,201,267 backtest windows.

5 Tokenization

The tokenizer turns each event into one integer ID. In plain terms, this is the market-data analogue of turning a word into a token ID in an LLM. The ID is formed from:

$$\texttt{action} \times \texttt{side} \times \texttt{relative\_price\_bucket} \times \texttt{price\_depth\_bucket} \times \texttt{size\_bucket} \times \texttt{time\_bucket}.$$

The submitted tokenizer uses quantile binning with a 0.01 clip quantile. The fitted vocabulary has 579 IDs, including special tokens. Realized bucket counts are action=2, side=2, relative_price=2, price_depth=3, size=8, and time=3. This choice keeps decoding auditable: every generated token maps back to an event-feature tuple rather than an opaque vector.

6 Model Architecture

`TradingModel` is a decoder-only Transformer: it reads a context window of prior event tokens and predicts the next event token. The implementation uses standard modern LLM components, including token embeddings, RoPE positional encoding, pre-norm decoder blocks, RMSNorm,

Figure 1: Mega-Trading data-to-model-to-output pipeline. The figure separates the data plane, tokenization, model core, training and evaluation, and inference output layers, with engineering evidence for lineage, custom kernels, and profiling connected back to the system modules they validate.

grouped-query causal attention, SwiGLU feed-forward layers, tied output embeddings, and a next-token cross entropy objective [5, 6].

The submitted single-GPU model uses hidden size 1024, 20 decoder layers, 16 query heads, 4 key-value heads, feed-forward width 2816, context length 512, and dropout 0. With the fitted vocabulary, this is approximately 226.1M tied-embedding trainable parameters. The size was chosen to be large enough to exercise a real Transformer training pipeline, but small enough to train and debug on one high-memory CUDA GPU. Grouped-query attention lowers key-value projection cost, RoPE handles causal sequence position without learned absolute position tables, and tied embeddings reduce parameters while preserving a simple token-language-model interface.

7 Training and Optimization

Training is orchestrated through Hydra and Hugging Face Accelerate. The submitted run uses a hybrid optimizer setup: AdamW for embeddings and related parameters, and Muon for large matrix parameters. It uses learning rate 0.0002 with cosine decay, 125 warmup steps, minimum learning rate 2×10^{-5}, batch size 64, mixed precision, Triton attention where available, and `torch.compile` [8].

Long single-GPU runs use prepared NumPy streams, persistent dataloader workers, pinned host transfer, non-blocking device transfer, a Triton causal GQA attention path, Triton RoPE and SwiGLU gate operators, and a Liger-Kernel-style fused linear-cross-entropy path that avoids materializing the full `[batch, time, vocab]` logits tensor on the loss head. The final submitted metric row reports 108K tokens/sec and a last-50-point mean of 111K tokens/sec. MLflow is enabled; the tracking database contains 9 runs, 596,177 metric rows, 273 params, and 72 tags.

8 Engineering Design Evidence

The repository contains design and profiling notes that should be read as part of the project evidence. They document decisions that are hard to infer from a single final metric table:

- `docs/data-plane-design.md` defines the Binance public-trades ingestion path, bounded-memory streaming prepare, tokenizer artifacts, and chronological split contract. This is the lineage layer behind every reported training and backtest metric.
- `docs/training-plane-design.md` records the NumPy shard input contract, checkpoint and manifest outputs, Accelerate-based DDP/FSDP execution path, and backtest handoff.
- `docs/kernels.md` documents the project's three custom Triton paths. The causal GQA attention forward kernel reports 156.23 approximate TFLOP/s versus 148.49 for PyTorch SDPA on the submitted RTX-shape bf16 benchmark; the note also records the current backward bottleneck (dK/dV register pressure and spills) so the limitation is explicit rather than hidden. The transformer operators (RoPE, SwiGLU gate, RMSNorm) ship a row-wise Triton implementation; RoPE and SwiGLU gate are faster than PyTorch on the target shape, RMSNorm is documented as an experiment but kept on PyTorch. A fused linear-cross-entropy kernel (Liger-Kernel-style: chunked cuBLAS GEMMs plus one row-wise online-softmax Triton kernel) cuts loss-head peak memory from 509 MB to 126 MB ($\sim$4.0$\times$) for the rtx training shape while staying within $\sim$12% of the PyTorch `linear` + `cross_entropy` latency. The same note records the failed hand-written fused-matmul variants (4–5$\times$ slower than cuBLAS) as a cautionary tuning history.
- `docs/performance-profiling.md` records the end-to-end PyTorch profiler investigation. The baseline CPU optimizer window was 323.645 ms over two optimizer updates; after

the shape-bucketed Muon optimization with `training.muon_ns_steps=3`, it dropped to about 20.018 ms.

- `docs/training-configs.md` explains the Mac, RTX, Modal, and default environments. The same pipeline can run a smoke test, single-server CUDA training, or Modal GPU training through config-driven entry points.
- `docs/training-systems-alignment.md` maps the implementation to training-system concerns: throughput, GPU utilization, memory pressure, I/O bottlenecks, checkpoint reliability, failure recovery, and learning per unit of compute.

9 Ablation Study

Hypotheses

The first ablation pass tested three practical hypotheses:

- **Model capacity**: a small decoder should outperform a micro decoder on the same one-ticker slice.
- **Data size**: adding a second ticker should make the task harder at the same short training budget.
- **Training budget**: more steps should improve likelihood, but may not automatically improve generated rollout statistics.

The probe used Binance monthly trades for BTCUSDT and ETHUSDT in April 2026, `block_size=128`, `stride=4096`, 16 backtest batches, and 5 decoded examples per run. The goal was to validate the ablation workflow and identify which dimensions are worth scaling next, not to claim a final leaderboard.

Table 1: Ablation probe results from `docs/ablation-results.md`.

Data	Model	Steps	Train	Val	Backtest	Top-1	L1
1 ticker	micro	20	3.410	2.915	3.060	0.619	0.632
1 ticker	small	20	2.580	1.922	2.114	0.621	0.146
2 tickers	micro	20	3.675	4.773	4.771	0.304	1.229
2 tickers	small	20	2.797	4.990	4.943	0.004	0.141
2 tickers	small	100	1.457	4.152	4.166	0.051	0.801

The cleanest conclusion is the one-ticker capacity result: moving from micro to small reduced validation loss from 2.915 to 1.922, backtest loss from 3.060 to 2.114, and price-depth L1 from 0.632 to 0.146. The data-size result is more subtle: adding ETHUSDT made next-token classification much harder at 20 steps, even when the small model preserved a plausible marginal price-depth distribution. The 100-step follow-up improved backtest loss from 4.943 to 4.166, but worsened price-depth L1 from 0.141 to 0.801. The practical selection rule is therefore to gate future runs on both likelihood and rollout realism.

10 Backtest Results

The submitted checkpoint was scored on 128 backtest batches, or 524,288 tokens.

Generated-versus-real stylized facts show that the model captures part of the held-out event distribution but still mismatches tail behavior. Real excess kurtosis is 3.0948; generated excess

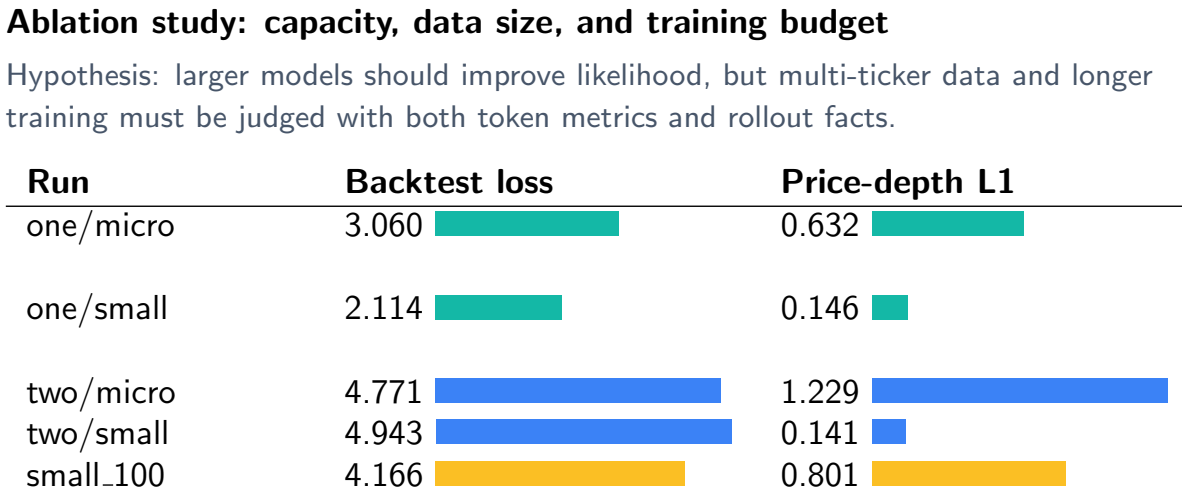

Figure 2: Ablation study summary across backtest loss, price-depth L1, and top-1 accuracy. The result shows why model selection should monitor both open-loop token metrics and rollout stylized facts.

Table 2: Open-loop chronological backtest metrics.

Metric	Value
Backtest loss	2.3379
Backtest perplexity	10.3592
Backtest top-1 accuracy	49.23%
Backtest top-5 accuracy	69.42%
Price-depth distribution L1	0.1420

kurtosis is 14.2966. That gap is a concrete target for better data, sampling, conditioning, and evaluation.

11 Accuracy Baselines

Top-1 accuracy should not be read against a vague random-guess baseline. With 579 realized token IDs, uniform random top-1 is only 0.17% and uniform random top-5 is 0.86%. The more serious issue is token imbalance: on the exact 524,288-token scored backtest sample, only 135 labels appear and the effective vocabulary is about 19.56.

Table 3: Prediction baselines on the same scored backtest sample.

Baseline	Top-1 / coverage
Uniform random top-1	0.17%
Uniform random top-5	0.86%
Cheating backtest majority token	25.41%
Cheating backtest top-5 frequency coverage	59.83%
Copy previous token	33.63%
Train-sample unigram top-1	2.10%
Train-sample unigram top-5	8.27%
Train-sample one-step Markov argmax	31.91%
Model top-1	49.23%
Model top-5	69.42%

The model is not worse than random and is meaningfully above copy-previous and one-step Markov baselines. However, the metric is still not sufficient evidence of trading usefulness because composite-token prediction is distribution-skewed. Top-k accuracy should be treated as a sanity metric.

12 Inference

Inference uses the same files produced during training:

1. Load `runs/rtx/checkpoint.pt`.
2. Load `datasets/mixture=rtx/tokenizer.json` and `tokens-profile.json`.
3. Validate the vocabulary size and context length.
4. Provide either a recent context window or a beginning-of-sequence seed.
5. Call `TradingModel.generate(input_ids, max_new_tokens, top_k=16)`.
6. Decode generated token IDs back into event-feature tuples.
7. Use decoded price-depth and side features for scenario charts or downstream simulation.

This makes inference reproducible and auditable: every generated scenario can be traced back through tokenizer metadata and model manifest fields.

13 Limitations and Roadmap

This is an open-loop public-data prototype. The current evaluation reports chronological backtest loss, perplexity, top-k token accuracy, and rollout-versus-real stylized facts. It does not yet implement the closed-loop market simulator emphasized by TradeFM, which would validate order matching, fills, queue dynamics, market impact, stress behavior, or execution-policy performance [3]. The public Binance trade adapter is also only a proxy for participant-observable order flow. Serious microstructure modeling should replace it with venue-grade feed data such as IEX DEEP/HIST market data, whose historical feed files are distributed in pcap format with feed specifications for decoding [7].

Future directions are:

- **L3 data**: build an IEX DEEP/HIST ingestion path for venue-grade historical market data, then extend the event contract toward add/delete/modify order lifecycle events, true best bid/ask midprice, queue position, and venue-specific order state [7].
- **Multimodal conditioning**: align news, filings, macro events, funding data, on-chain or flow signals, and sentiment with event-time order-flow tokens.
- **Closed-loop simulator**: evaluate generated events inside a deterministic LOB simulator to measure fills, slippage, spread dynamics, and market impact.
- **Inference packaging**: ship checkpoint, tokenizer, manifest, and model card together so generated scenarios preserve provenance.
- **Evaluation expansion**: add regime-conditioned metrics, tail-risk tests, execution-policy benchmarks, and calibration checks.

References

[1] Deeter Analytics Inc. Company website. https://deeteranalytics.com/.

[2] Binance. Binance Public Data. https://data.binance.vision/.

[3] TradeFM. *TradeFM: A Generative Foundation Model for Trade-flow and Market Microstructure*. arXiv:2602.23784, 2026.

[4] Rishi Bommasani et al. *On the Opportunities and Risks of Foundation Models*. arXiv:2108.07258, 2021.

[5] Ashish Vaswani et al. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 2017.

[6] AI at Meta, Llama Team. *The Llama 3 Herd of Models*. arXiv:2407.21783, 2024.

[7] IEX Exchange. Market Data and HIST Download. https://iextrading.com/trading/market-data/.

[8] Philippe Tillet, H. T. Kung, and David Cox. *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations*. MAPL, 2019. https://doi.org/10.1145/3315508.3329973.